

**e**mbarcadero®

# EMBARCADERO CONFERENCE 2025

A JORNADA DO HERÓI | 30 ANOS DE INOVAÇÃO





# Desenvolvimento de Aplicações Desktop de Alta Performance

Ivan de Souza

Serão abordados conceitos fundamentais de desempenho em sistemas desktop, boas práticas com VCL e FireMonkey, gerenciamento de memória, uso de TThread para manter a interface responsiva e otimizações com FireDAC. Também exploraremos API's nativas do Windows (GDI) para acelerar operações gráficas e ferramentas como TMS MemInsight para identificar gargalos de performance.



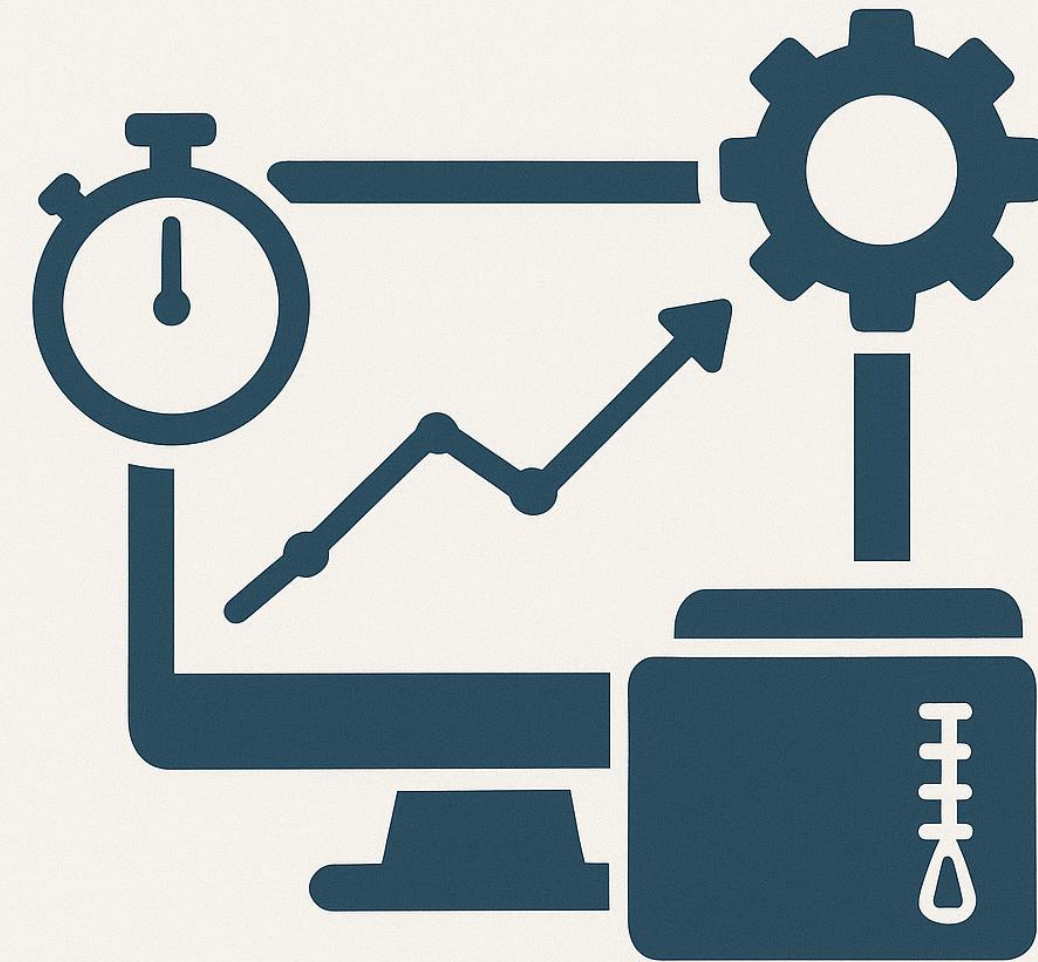
# 1. Introdução

## O que é desempenho?

- Capacidade de executar tarefas com rapidez e eficiência

## Indicadores

- Tempo de resposta
- Uso de CPU e memória
- **Por que importa?**
  - Melhor experiência do usuário
  - Escalabilidade sustentável





## 2. Gerenciamento Eficiente de Memória

### Problema:

- Vazamentos → apps lentos e instáveis

- **Solução:**

- Uso de try..finally
- Liberação correta de objetos (Free, FreeAndNil)
- Ferramenta: **FastMM** para detectar vazamentos

### Benefício:

- Uso eficiente e previsível dos recursos





### 3. Uso de Threads para Tarefas Pesadas

#### Problema:

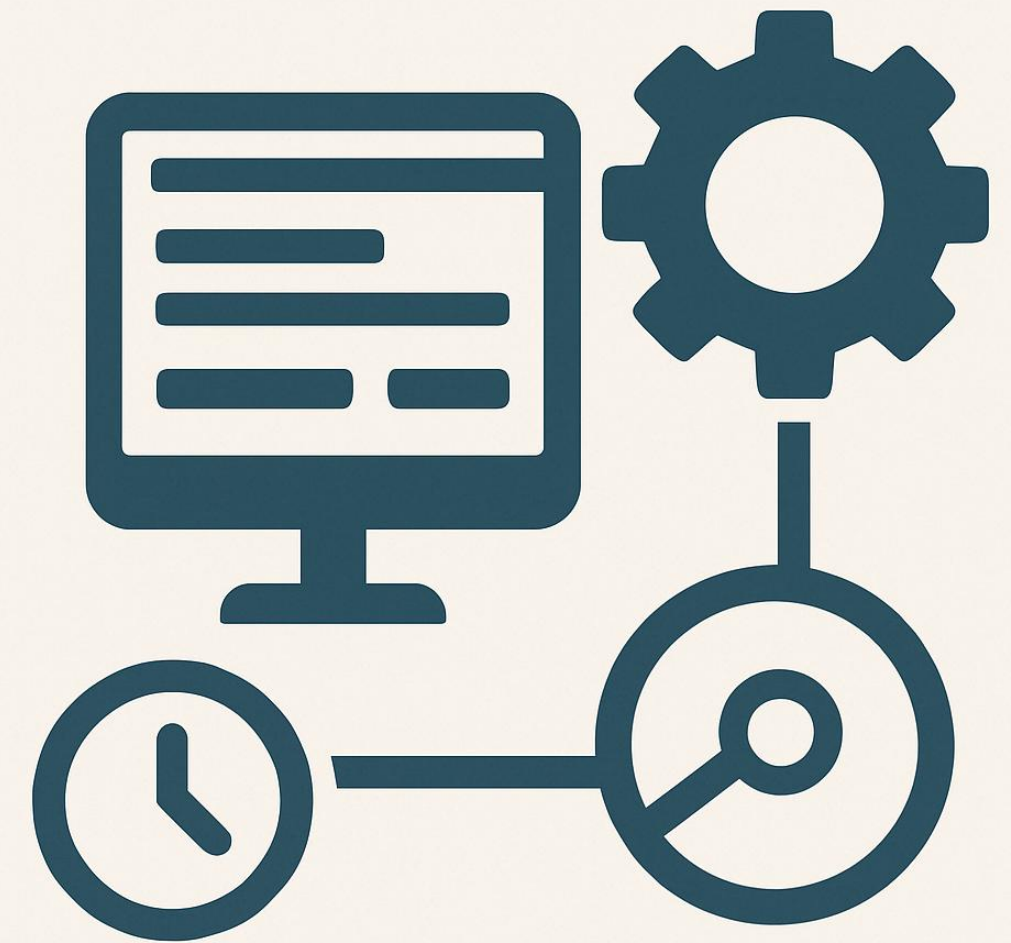
- Tarefas pesadas travam a interface (UI)

#### Solução:

- TThread para processamento paralelo
- Synchronize para atualizar UI com segurança
- FreeOnTerminate para liberar memória da thread

#### Alternativa moderna:

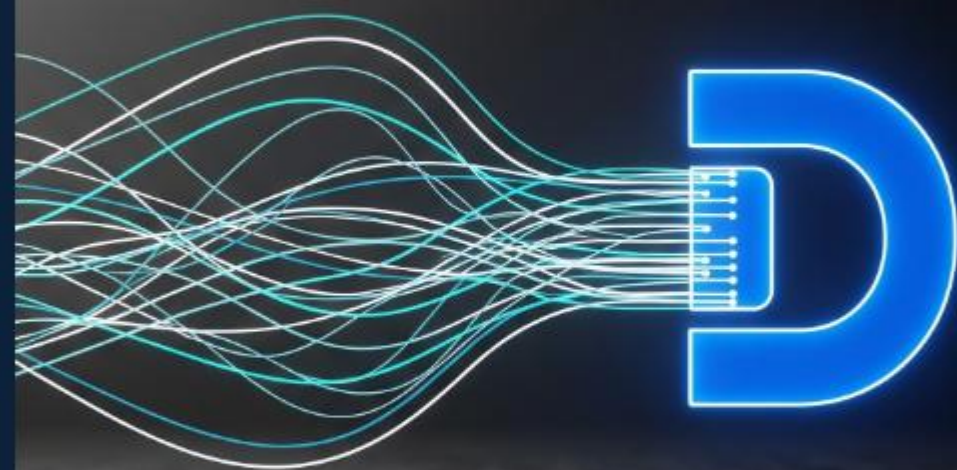
- Parallel Programming Library (PPL)





# Delphi Multithreading

Threads, Concorrência, Paralelismo e Assincronismo



Cesar Romero

## A Elegância da Parallel Programming Library (PPL)

Modernize seu código com `TTask`, `IFuture` e `TParallel.For`. Orquestre fluxos de trabalho complexos, processe dados em paralelo e execute requisições de rede de forma elegante, escrevendo menos código e alcançando mais performance.

## Padrões de Arquitetura e Código Limpo

Uma aplicação *multithreaded* robusta começa com uma boa arquitetura. Aprenda as melhores práticas para organizar seu código, desacoplar a lógica de negócio da UI e criar um código limpo, testável e de fácil manutenção.

## Sincronização Desmistificada

Domine `TCriticalSection`, `TMutex`, `TMonitor`, `TEvent` e `TSemaphore`. Aprenda não apenas *como* usá-las, mas *quando* e *porquê*, com exemplos práticos para prevenir *Race Conditions* e *Deadlocks*.

## Crie Apps Mobile Realmente Responsivos

Este livro dedica um módulo inteiro às **particularidades da concorrência em plataformas mobile**. Aprenda a evitar os temidos erros de "Application Not Responding" (ANRs) no Android e a garantir uma experiência de usuário fluida.

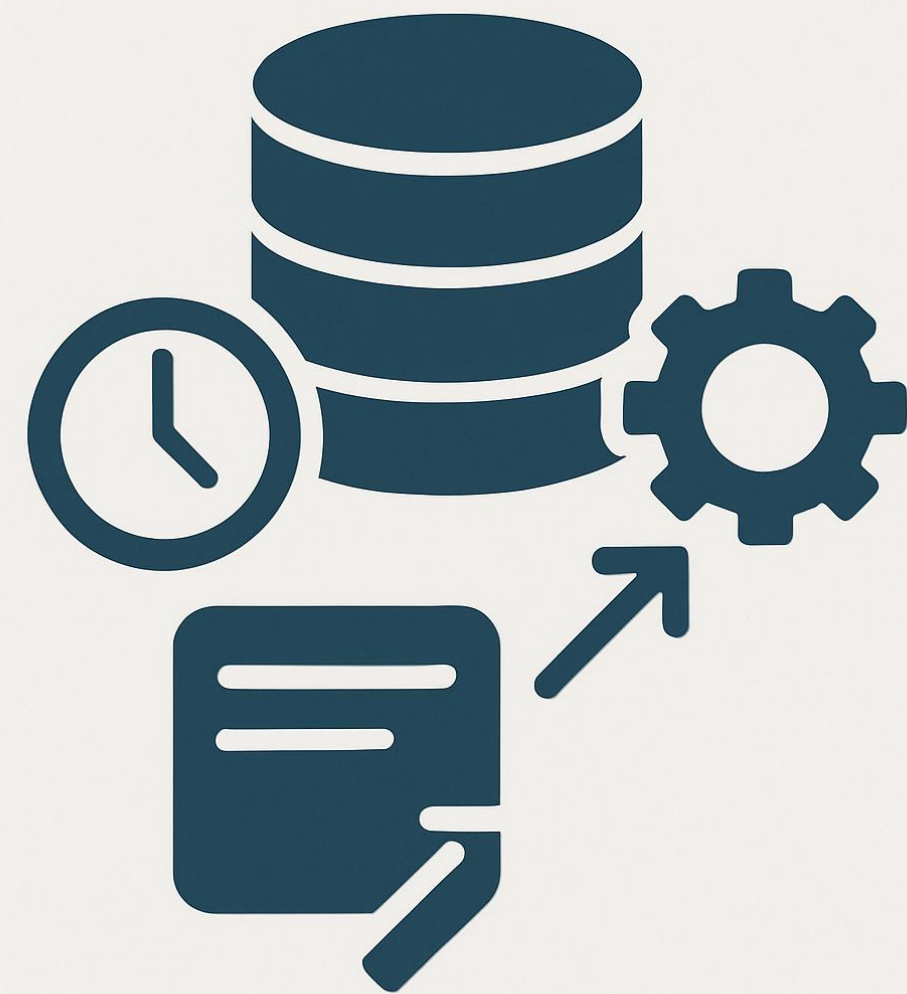
[www.cesarromero.com.br](http://www.cesarromero.com.br)



## 4. Otimização de Acesso a Bancos de Dados

### Boas práticas:

- Consultas parametrizadas
  - Paginação com LIMIT / OFFSET
  - Carregamento sob demanda (OnDemand)
- 
- **Benefícios:**
    - Menor uso de memória
    - Maior velocidade nas consultas





## 5. Aproveitando Recursos do Sistema Operacional

### APIs nativas do Windows:

- GDI para gráficos
- **CreateFile** para operações de I/O
- **Vantagens:**
  - Desempenho superior
  - Melhor integração com o SO





# 6. Ferramentas de Perfilamento e Depuração

Descobrir gargalos:

- CPU, memória, I/O
- Ferramentas:
  - TMS MemInsight
  - AQTime
  - Depurador do Delphi
- Dica extra:
  - Configuração do compilador {\$O+} para otimizações

tmssoftware.com

PRODUCTS

Home / Products / Developer Tools / TMS MemInsight

DEV

TMS MemInsight

Delphi run-time memory allocation inspection, exception call stack logging, thread inspection and more

The screenshot displays the TMS MemInsight 1.0.0 application window. The main window has a title bar "tms: TMS MemInsight 1.0.0 MainThreadID: 16080" and a menu bar with "Memory", "Statistics", "Threads", "Exceptions", and "Modules". Below the menu bar is a table with columns: ID, Address, Size, Thread, Type, and Data. The table lists memory allocations, with the selected row (ID 1395) showing a TButton object at address \$026FF9C0, size 764, allocated on the MainThread. Below the table, it shows "32 blocks, 2.960 bytes (32 objects, 0 strings, 0 arrays, 0 records)". At the bottom are buttons for "Update", "Clear", "Snapshot", and "Options".

A secondary window titled "tms: Memory Details" is open, showing a call stack. It has tabs for "Details", "CallStack", "Properties", "Inheritance", and "Dump". The "CallStack" tab is active, showing a list of addresses, modules, and procedures. The selected entry is at address \$006D7C1B, module UDemo, and procedure TForm1.ButtonLeakButtonClick (197).

| ID   | Address    | Size | Thread     | Type     | Data                          |
|------|------------|------|------------|----------|-------------------------------|
| 50   | \$0274A6A0 | 24   | MainThread | mtObject | System.Classes.TMemoryStream  |
| 56   | \$0274A9A0 | 24   | MainThread | mtObject | System.Classes.TMemoryStream  |
| 1395 | \$026FF9C0 | 764  | MainThread | mtObject | Vcl.StdCtrls.TButton          |
| 1396 | \$02770080 | 44   | MainThread | mtObject | Vcl.Graphics.TFont            |
| 1397 | \$06FB07E8 | 36   | MainThread | mtObject | Vcl.Controls.TSizeConstraints |
| 1398 | \$06FB0810 | 36   | MainThread | mtObject |                               |
| 1399 | \$06FB0860 | 36   | MainThread | mtObject |                               |
| 1400 | \$02754348 | 20   | MainThread | mtObject |                               |
| 1401 | \$0274F6C0 | 28   | MainThread | mtObject |                               |
| 1402 | \$06FB0900 | 36   | MainThread | mtObject |                               |
| 1403 | \$06FB0950 | 36   | MainThread | mtObject |                               |
| 1404 | \$06FB09A0 | 36   | MainThread | mtObject |                               |
| 3355 | \$02670450 | 980  | MainThread | mtObject |                               |
| 3356 | \$02770050 | 44   | MainThread | mtObject |                               |
| 3357 | \$06FB09F0 | 36   | MainThread | mtObject |                               |
| 3358 | \$06FB0A40 | 36   | MainThread | mtObject |                               |
| 3359 | \$06FB0A68 | 36   | MainThread | mtObject |                               |
| 3360 | \$02754378 | 20   | MainThread | mtObject |                               |

| Address    | Module         | Procedure                          |
|------------|----------------|------------------------------------|
| \$00557320 | Vcl.Controls   | TWinControl.Create (8512)          |
| \$0057769F | Vcl.StdCtrls   | TButtonControl.Create (1919)       |
| \$006D7C1B | UDemo          | TForm1.ButtonLeakButtonClick (197) |
| \$0055DD9  | Vcl.Controls   | TControl.Click (7537)              |
| \$0055A334 | Vcl.Controls   | TWinControl.WndProc (10280)        |
| \$005777DD | Vcl.StdCtrls   | TButtonControl.WndProc (1919)      |
| \$0055A498 | Vcl.Controls   | DoControlMsg (10347)               |
| \$0055A334 | Vcl.Controls   | TWinControl.WndProc (10280)        |
| \$005598FF | Vcl.Controls   | TWinControl.MainWndProc (9977)     |
| \$0048451A | System.Classes | StdWndProc (4324)                  |



# 7. Conclusão e Boas Práticas

## Resumo:

- Memória, Threads, Banco de dados, APIs do SO, Profiling
- **Boas práticas:**
  - Código limpo
  - Modularidade
  - Testes unitários (DUnit)





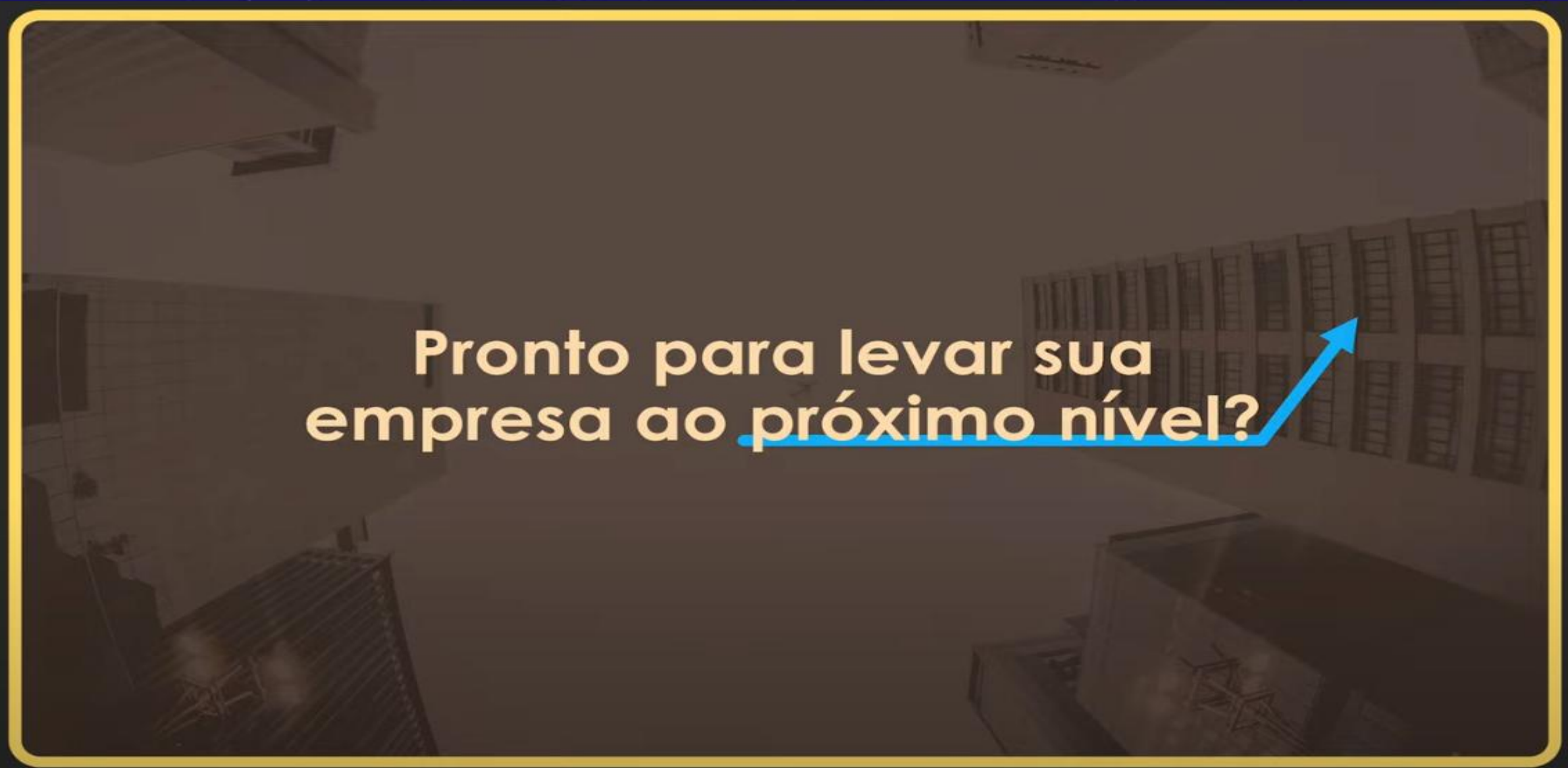
# E após alguns slides ...

Quero expressar minha gratidão pela oportunidade de compartilhar conhecimento.

Cada troca gera aprendizado e fortalece o crescimento mútuo. Agradeço pelo tempo dedicado e pela confiança no processo.

Que cada passo dado seja repleto de conquistas e evolução.

## Então ...



Pronto para levar sua  
empresa ao próximo nível?



embarcadero®

# EMBARCADERO CONFERENCE 2025

A JORNADA DO HERÓI | 30 ANOS DE INOVAÇÃO



Quer me ver na  
**#ECON26?**  
Acesse o QRCode e  
avalie minha palestra!



**Ivan Souza**

 @ivansouza2012

 /ivanlsouza

 ilsouza@gmail.com

 (43) 9 9102-5152